

Sistem de recomandare pentru jocuri video

Aplicație bazată pe grafuri folosind Neo4j și FastAPI

Nume student: Iacob Delcea

Rezumat

Această lucrare prezintă proiectarea și implementarea unui sistem de recomandare pentru jocuri video folosind baza de date orientată pe grafuri Neo4j și framework-ul FastAPI. Aplicația modelează utilizatori, jocuri, genuri, platforme, developeri și publisheri sub forma unui graf, iar recomandările sunt generate prin traversarea relațiilor dintre aceste entități. Scopul proiectului este demonstrarea avantajelor unei baze de date graf în scenarii unde relațiile dintre date sunt importante, cum ar fi recomandările, utilizatorii similari și identificarea jocurilor asemănătoare.

Cuprins

1	Introducere	3
1.1	Obiectivele proiectului	3
2	Baze de date relaționale vs. baze de date graf	4
2.1	Baze de date relaționale	4
2.2	Baze de date graf	4
2.3	Comparație SQL - Neo4j	5
2.4	Când nu este potrivit Neo4j	5
3	Tehnologii folosite	6
3.1	Neo4j	6
3.2	Cypher	6
3.3	FastAPI	7
3.4	Neo4j Python Driver	7
3.5	Docker Compose	7
4	Modelul de date	8
4.1	Noduri	8
4.2	Relații	8
4.3	Exemplu concret	9
5	Arhitectura aplicației	10
5.1	Structura proiectului	10
5.2	Rolul componentelor	11
5.3	Fluxul unei cereri	11
6	Endpointuri API	12
6.1	Endpointuri pentru utilizatori	12
6.2	Endpointuri pentru jocuri	12
6.3	Endpointuri pentru recomandări	12
6.4	Endpointuri pentru statistici și performanță	13

7	Interogări Cypher importante	14
7.1	Crearea unui utilizator	14
7.2	Crearea unui joc și a relațiilor sale	14
7.3	Recomandări pe baza genurilor apreciate	15
7.4	Utilizatori similari	15
7.5	Jocuri similare	15
8	Statistici	16
8.1	Exemplu de statistici generale	16
8.2	Statistici pe genuri	16
9	Analiza performanței	17
9.1	Scopul testelor de performanță	17
9.2	Indecși folosiți	17
9.3	Exemplu de rezultate	18
9.4	Interpretare	18
10	Instrucțiuni de rulare	19
10.1	Pornirea bazei de date	19
10.2	Configurarea aplicației	19
10.3	Instalarea dependențelor	19
10.4	Popularea bazei de date	20
10.5	Pornirea API-ului	20
11	Concluzii	21

Capitolul 1

Introducere

În aplicațiile moderne, datele nu sunt importante doar prin valorile lor individuale, ci și prin relațiile dintre ele. Un exemplu relevant este un sistem de recomandare, unde deciziile nu se bazează doar pe lista de jocuri existente, ci și pe conexiunile dintre utilizatori, jocuri, genuri, platforme și preferințe.

Proiectul propus implementează un sistem de recomandare pentru jocuri video. Utilizatorii pot juca, aprecia și evalua jocuri, iar fiecare joc este conectat cu genuri, platforme, dezvoltari și publisheri. Pe baza acestor conexiuni, aplicația poate recomanda jocuri similare cu cele apreciate anterior de un utilizator.

Neo4j este potrivit pentru acest tip de aplicație deoarece folosește un model de date bazat pe grafuri. Conform documentației Neo4j, o bază de date graf este formată din noduri, relații și căi, iar limbajul Cypher este folosit pentru a crea și interoga aceste elemente [1]. FastAPI este folosit pentru expunerea endpointurilor REST, fiind un framework modern pentru construirea API-urilor pe baza type hint-urilor standard [2].

1.1 Obiectivele proiectului

Obiectivele principale ale proiectului sunt:

- modelarea datelor sub formă de graf;
- implementarea unei aplicații backend folosind FastAPI;
- integrarea aplicației cu Neo4j prin driverul oficial Python;
- crearea unor endpointuri pentru utilizatori, jocuri, recomandări și statistici;
- generarea de date de test printr-un script de seed;
- analiza avantajelor și limitărilor Neo4j față de o bază de date relațională SQL;
- realizarea unor teste simple de performanță.

Capitolul 2

Baze de date relaționale vs. baze de date graf

2.1 Baze de date relaționale

Bazele de date relaționale organizează datele în tabele. Relațiile dintre entități sunt reprezentate prin chei primare și chei externe. Acest model este foarte eficient pentru aplicații cu date structurate, relații stabile și operații de tip CRUD, raportare sau agregare.

Exemple de aplicații potrivite pentru SQL:

- aplicații de contabilitate;
- aplicații bancare;
- aplicații de pontaj;
- magazine online clasice.

2.2 Baze de date graf

Bazele de date graf modelează datele ca noduri și relații. În loc să fie necesare multe operații de tip JOIN, conexiunile sunt reprezentate direct în baza de date. Acest lucru este util atunci când aplicația trebuie să răspundă la întrebări de tipul:

- Ce entități sunt conectate între ele?
- Ce jocuri sunt similare cu un anumit joc?
- Ce utilizatori au preferințe asemănătoare?
- Ce recomandări pot fi generate pe baza relațiilor existente?

2.3 Comparație SQL - Neo4j

Tabela 2.1: Comparație între SQL și Neo4j

Caracteristică	SQL	Neo4j
Model de date	Tabele, rânduri, coloane	Noduri, relații, proprietăți
Limbaaj de interogare	SQL	Cypher
Relații	Chei externe și JOIN-uri	Muchii directe între noduri
Potrivit pentru	Date tabulare, rapoarte, tranzacții	Date foarte conectate, recomandări, grafuri
Performanță pe relații multiple	Poate necesita multe JOIN-uri	Traversare directă prin relații
Spațiu de stocare	De obicei mai compact pentru date simple	Poate avea overhead pentru noduri și relații

2.4 Când nu este potrivit Neo4j

Neo4j nu este alegerea ideală atunci când datele sunt predominant tabelare, relațiile sunt simple și operațiile principale sunt filtrări, sortări, agregări și rapoarte. În aceste situații, SQL poate fi mai eficient ca viteză, mai compact ca spațiu de stocare și mai simplu de administrat.

De exemplu, pentru o aplicație de facturare unde principalele operații sunt calculul totalului vânzărilor, listarea comenzilor și generarea de rapoarte lunare, SQL este de obicei mai potrivit. Neo4j devine avantajos atunci când relațiile dintre date sunt la fel de importante ca datele în sine.

Capitolul 3

Tehnologii folosite

3.1 Neo4j

Neo4j este baza de date folosită pentru stocarea grafului. În acest proiect, Neo4j stochează utilizatorii, jocurile, genurile, platformele, developerii și publisherii. Relațiile dintre aceste noduri sunt folosite pentru generarea recomandărilor.

Principalele concepte Neo4j utilizate sunt:

- **Node** - entitate din graf, de exemplu **User** sau **Game**;
- **Relationship** - legătură între două noduri, de exemplu **LIKED**;
- **Label** - categorie a unui nod, de exemplu **:User**;
- **Property** - atribut al unui nod sau al unei relații;
- **Path** - succesiune de noduri și relații.

3.2 Cypher

Cypher este limbajul folosit pentru interogarea bazei Neo4j. Acesta permite descrierea intuitivă a tiparelor din graf. De exemplu, pentru a găsi jocurile apreciate de un utilizator se poate folosi următoarea interogare:

```
MATCH (u:User {id: $user_id})-[:LIKED]->(g:Game)
RETURN g
```

Listing 3.1: Jocuri apreciate de un utilizator

3.3 FastAPI

FastAPI este framework-ul folosit pentru dezvoltarea backendului. Acesta permite definirea rapidă a endpointurilor REST, validarea datelor de intrare și generarea automată a documentației Swagger. FastAPI este descris în documentația oficială ca un framework modern și performant pentru construirea API-urilor cu Python [2].

3.4 Neo4j Python Driver

Aplicația comunică cu baza de date prin driverul oficial Neo4j pentru Python [3]. Acesta permite crearea unei conexiuni la baza de date și executarea interogărilor Cypher din codul Python.

3.5 Docker Compose

Neo4j este rulat local folosind Docker Compose. Docker Compose permite definirea și rularea aplicațiilor multi-container printr-un fișier YAML de configurare [4]. În proiect, acesta este folosit pentru pornirea rapidă a containerului Neo4j.

Capitolul 4

Modelul de date

4.1 Noduri

Modelul de date conține următoarele tipuri de noduri:

Tabela 4.1: Nodurile aplicației

Nod	Descriere
User	Reprezintă un utilizator al aplicației.
Game	Reprezintă un joc video.
Genre	Reprezintă genul unui joc, de exemplu RPG, Action sau Adventure.
Platform	Reprezintă platforma pe care este disponibil jocul, de exemplu PC sau PlayStation 5.
Developer	Reprezintă studioul care a dezvoltat jocul.
Publisher	Reprezintă compania care a publicat jocul.

4.2 Relații

Relațiile sunt esențiale pentru funcționarea aplicației, deoarece recomandările sunt generate prin traversarea lor.

Tabela 4.2: Relațiile aplicației

Relație	Descriere
(User) - [:PLAYED] -> (Game)	Utilizatorul a jucat un joc.
(User) - [:LIKED] -> (Game)	Utilizatorul a apreciat un joc.
(User) - [:RATED] -> (Game)	Utilizatorul a evaluat un joc. Relația conține proprietatea score .
(Game) - [:HAS_GENRE] -> (Genre)	Jocul aparține unui gen.
(Game) - [:AVAILABLE_ON] -> (Platform)	Jocul este disponibil pe o anumită platformă.
(Game) - [:DEVELOPED_BY] -> (Developer)	Jocul a fost dezvoltat de un anumit studio.
(Game) - [:PUBLISHED_BY] -> (Publisher)	Jocul a fost publicat de o anumită companie.

4.3 Exemplu concret

Un exemplu de date din sistem este:

- User: iacob
- Game: Elden Ring
- Genre: RPG, Soulslike, Open World
- Platform: PC, PlayStation 5, Xbox Series X
- Developer: FromSoftware
- Publisher: Bandai Namco

Relațiile corespunzătoare pot fi reprezentate astfel:

```
(:User {username: "iacob"}) -[:PLAYED] -> (:Game {title: "Elden Ring"
})
(:User {username: "iacob"}) -[:LIKED] -> (:Game {title: "Elden Ring"})
(:Game {title: "Elden Ring"}) -[:HAS_GENRE] -> (:Genre {name: "RPG"})
(:Game {title: "Elden Ring"}) -[:AVAILABLE_ON] -> (:Platform {name: "
PC"})
(:Game {title: "Elden Ring"}) -[:DEVELOPED_BY] -> (:Developer {name: "
FromSoftware"})
(:Game {title: "Elden Ring"}) -[:PUBLISHED_BY] -> (:Publisher {name: "
Bandai Namco"})
```

Listing 4.1: Exemplu de relații în graf

Capitolul 5

Arhitectura aplicației

5.1 Structura proiectului

Aplicația este organizată modular, pentru a separa responsabilitățile fiecărei componente.

```
neo4j-game-recommender /
|
|-- app/
|   |-- main.py
|   |-- config.py
|   |-- db/
|       |-- neo4j_client.py
|   |-- models/
|       |-- user.py
|       |-- game.py
|   |-- routers/
|       |-- users.py
|       |-- games.py
|       |-- recommendations.py
|       |-- statistics.py
|   |-- services/
|       |-- user_service.py
|       |-- game_service.py
|       |-- recommendation_service.py
|       |-- statistics_service.py
|       |-- performance_service.py
|   |-- seed/
|       |-- seed_data.py
|
|-- docker-compose.yml
|-- requirements.txt
```

5.2 Rolul componentelor

Tabela 5.1: Componentele aplicației

Componentă	Rol
<code>main.py</code>	Inițializează aplicația FastAPI și include routerele.
<code>config.py</code>	Citește variabilele de configurare din environment variables.
<code>neo4j_client.py</code>	Creează conexiunea la Neo4j și execută query-uri Cypher.
<code>models/</code>	Conține modelele Pydantic pentru requesturi și responseuri.
<code>routers/</code>	Definește endpointurile API.
<code>services/</code>	Conține logica aplicației și query-urile către Neo4j.
<code>seed_data.py</code>	Populează baza de date cu date inițiale.

5.3 Fluxul unei cereri

Pentru endpointul:

```
GET /users/{user_id}/recommendations
```

fluxul este următorul:

1. Clientul trimite cererea HTTP către API.
2. FastAPI primește cererea în routerul de recomandări.
3. Routerul apelează metoda corespunzătoare din `RecommendationService`.
4. Serviciul construiește și execută un query Cypher.
5. Neo4j returnează jocurile recomandate.
6. FastAPI trimite rezultatul către client în format JSON.

Capitolul 6

Endpointuri API

6.1 Endpointuri pentru utilizatori

Tabela 6.1: Endpointuri pentru utilizatori

Metodă	Endpoint	Descriere
POST	/users	Creează un utilizator nou.
GET	/users	Returnează lista utilizatorilor.
GET	/users/{user_id}	Returnează un utilizator după ID.
POST	/users/{user_id}/play/{game_id}	Marchează un joc ca jucat.
POST	/users/{user_id}/like/{game_id}	Marchează un joc ca apreciat.
POST	/users/{user_id}/rate/{game_id}	Adaugă sau actualizează ratingul unui joc.

6.2 Endpointuri pentru jocuri

Tabela 6.2: Endpointuri pentru jocuri

Metodă	Endpoint	Descriere
POST	/games	Creează un joc nou și relațiile sale.
GET	/games	Returnează lista jocurilor.
GET	/games/{game_id}	Returnează un joc după ID.

6.3 Endpointuri pentru recomandări

Tabela 6.3: Endpointuri pentru recomandări

Metodă	Endpoint	Descriere
GET	/users/{user_id}/recommendations	Recomandă jocuri pentru un utilizator.
GET	/users/{user_id}/similar-users	Returnează utilizatori cu preferințe similare.
GET	/games/{game_id}/similar	Returnează jocuri similare cu un joc dat.

6.4 Endpointuri pentru statistici și performanță

Tabela 6.4: Endpointuri pentru statistici și performanță

Metodă	Endpoint	Descriere
GET	/statistics	Returnează statistici generale despre graf.
POST	/indexes	Creează indecși pentru nodurile principale.
GET	/performance/test	Execută teste simple de performanță.

Capitolul 7

Interogări Cypher importante

7.1 Crearea unui utilizator

```
MERGE (u:User {id: $id})
SET u.username = $username,
    u.age = $age
RETURN u
```

Listing 7.1: Creare utilizator

7.2 Crearea unui joc și a relațiilor sale

```
MERGE (g:Game {id: $id})
SET g.title = $title,
    g.release_year = $release_year

FOREACH (genreName IN $genres |
    MERGE (genre:Genre {name: genreName})
    MERGE (g)-[:HAS_GENRE]->(genre)
)

FOREACH (platformName IN $platforms |
    MERGE (platform:Platform {name: platformName})
    MERGE (g)-[:AVAILABLE_ON]->(platform)
)
```

Listing 7.2: Creare joc cu genuri și platforme

7.3 Recomandări pe baza genurilor apreciate

Această interogare pornește de la jocurile apreciate de utilizator, identifică genurile acelor jocuri și caută alte jocuri care au aceleași genuri.

```
MATCH (u:User {id: $user_id})-[:LIKED]->(Game)-[:HAS_GENRE]->(
    genre:Genre)
MATCH (recommended:Game)-[:HAS_GENRE]->(genre)
WHERE NOT (u)-[:LIKED]->(recommended)
      AND NOT (u)-[:PLAYED]->(recommended)
RETURN recommended, count(DISTINCT genre) AS score
ORDER BY score DESC
LIMIT $limit
```

Listing 7.3: Recomandări pe baza genurilor

7.4 Utilizatori similari

Utilizatorii similari sunt identificați pe baza jocurilor apreciate în comun.

```
MATCH (u1:User {id: $user_id})-[:LIKED]->(g:Game)<-[:LIKED]-(u2:
    User)
WHERE u1 <> u2
RETURN u2, count(g) AS common_likes
ORDER BY common_likes DESC
LIMIT $limit
```

Listing 7.4: Utilizatori similari

7.5 Jocuri similare

Jocurile similare sunt identificate pe baza genurilor comune.

```
MATCH (g1:Game {id: $game_id})-[:HAS_GENRE]->(genre:Genre)<-[:
    HAS_GENRE]-(g2:Game)
WHERE g1 <> g2
RETURN g2, count(DISTINCT genre) AS score
ORDER BY score DESC
LIMIT $limit
```

Listing 7.5: Jocuri similare

Capitolul 8

Statistici

Aplicația include un endpoint dedicat pentru statistici. Acesta poate returna numărul total de utilizatori, jocuri, genuri, platforme și relații.

8.1 Exemplu de statistici generale

Tabela 8.1: Exemplu de statistici generale

Metrică	Valoare exemplu
Număr utilizatori	5
Număr jocuri	10
Număr genuri	8
Număr platforme	5
Relații PLAYED	16
Relații LIKED	11
Relații RATED	8

8.2 Statistici pe genuri

Tabela 8.2: Exemplu de statistici pe genuri

Gen	Număr jocuri
RPG	4
Open World	4
Action	5
Adventure	4
Soulslike	2

Capitolul 9

Analiza performanței

9.1 Scopul testelor de performanță

Testele de performanță au rolul de a observa timpul de execuție al unor interogări importante din aplicație. Endpointul `/performance/test` măsoară timpul pentru:

- căutarea unui utilizator după ID;
- căutarea unui joc după ID;
- generarea recomandărilor pe baza genurilor;
- identificarea utilizatorilor similari.

9.2 Indecși folosiți

Pentru îmbunătățirea performanței, aplicația creează indecși pe proprietățile folosite frecvent în căutări.

```
CREATE INDEX user_id_index IF NOT EXISTS
FOR (u:User)
ON (u.id)

CREATE INDEX game_id_index IF NOT EXISTS
FOR (g:Game)
ON (g.id)

CREATE INDEX genre_name_index IF NOT EXISTS
FOR (g:Genre)
ON (g.name)

CREATE INDEX platform_name_index IF NOT EXISTS
```

```
FOR (p:Platform)
ON (p.name)
```

Listing 9.1: Indecși pentru optimizare

9.3 Exemplu de rezultate

Tabela 9.1: Exemplu de rezultate pentru testele de performanță

Interogare	Timp execuție	Înregistrări returnate
Find user by id	6.4 ms	1
Find game by id	4.6 ms	1
Recommendations by genre	5.7 ms	7
Similar users	4.4 ms	2

Valorile de mai sus sunt orientative. În practică, ele pot depinde de hardware, de volumul de date, de existența indecșilor și de starea cache-ului intern al bazei de date.

9.4 Interpretare

Căutările directe după ID sunt rapide atunci când există indecși. Interogările de recomandare sunt mai complexe deoarece traversează mai multe relații, însă acestea sunt exact tipul de operații pentru care o bază de date graf este potrivită.

Într-o bază de date relațională, același scenariu ar necesita mai multe tabele intermediare și mai multe operații de tip JOIN. În Neo4j, relațiile sunt stocate explicit și pot fi parcurse natural prin Cypher.

Capitolul 10

Instrucțiuni de rulare

10.1 Pornirea bazei de date

Pentru pornirea bazei Neo4j se folosește comanda:

```
docker compose up -d
```

Neo4j Browser poate fi accesat la:

```
http://localhost:7474
```

Credentialele locale folosite în proiect sunt:

```
username: neo4j
password: password123
```

10.2 Configurarea aplicației

Aplicația citește datele de configurare din variabile de mediu. În PyCharm, acestea se pot seta în *Run Configuration*, la câmpul *Environment variables*.

```
NEO4J_URI=bolt://localhost:7687;
NEO4J_USERNAME=neo4j;
NEO4J_PASSWORD=password123;
NEO4J_DATABASE=neo4j;
API_HOST=127.0.0.1;
API_PORT=8000
```

10.3 Instalarea dependențelor

```
pip install -r requirements.txt
```

10.4 Popularea bazei de date

```
python -m app.seed.seed_data
```

10.5 Pornirea API-ului

API-ul poate fi pornit direct din fișierul `main.py` sau folosind comanda:

```
python app/main.py
```

Documentația Swagger este disponibilă la:

```
http://127.0.0.1:8000/docs
```

Capitolul 11

Concluzii

Proiectul demonstrează modul în care Neo4j poate fi folosit pentru construirea unui sistem de recomandare bazat pe relații. Modelarea datelor sub formă de graf este naturală pentru domeniul jocurilor video, deoarece există multe conexiuni între utilizatori, jocuri, genuri, platforme și companii.

FastAPI oferă o metodă simplă și eficientă de expunere a funcționalităților prin endpointuri REST, iar Neo4j permite interogări expresive prin Cypher. Recomandările, utilizatorii similari și jocurile similare sunt obținute prin traversarea relațiilor din graf, ceea ce evidențiază avantajul unei baze de date graf în comparație cu o bază relațională folosită pentru același tip de problemă.

În același timp, proiectul arată că Neo4j nu este potrivit pentru toate scenariile. Pentru aplicații cu date predominant tabelare, rapoarte și relații simple, SQL rămâne o alegere mai bună. Alegerea tehnologiei trebuie făcută în funcție de tipul datelor și de întrebările la care aplicația trebuie să răspundă.

Bibliografie

- [1] Neo4j, *Cypher Manual - Core concepts*. Disponibil online: <https://neo4j.com/docs/cypher-manual/current/queries/concepts/>
- [2] FastAPI, *FastAPI Documentation*. Disponibil online: <https://fastapi.tiangolo.com/>
- [3] Neo4j, *Neo4j Python Driver Manual*. Disponibil online: <https://neo4j.com/docs/api/python-driver/current/>
- [4] Docker, *Docker Compose Documentation*. Disponibil online: <https://docs.docker.com/compose/>